D1861R0
Secure Connections in Networking TS

Draft Proposal, 2018-09-05

This version:
http://wg21.link/P1861R0

Authors:
Alex Christensen (Apple)
JF Bastien (Apple)

Audience:
LEWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:
https://github.com/achristensen07/papers/blob/master/source/p1861r0.bs

---

Table of Contents

§ 1. Abstract

This paper shows a minimal change to the existing [N4771] Networking TS to support TLS and DTLS.

§ 2. Introduction

In [P1860R0] we make the case that C++ networking should be secure by default, motivating the addition of TLS and DTLS support to the Networking TS. This paper describes minimal changes necessary to implement secure connections for use on the internet.

This paper does not claim to contain everything that would be required to support secure connections, but rather it is a glimpse into what it would look like if we decided to take the existing Networking TS and add security without any further reshaping.

The changes in this paper are not intended to be accepted by the C++ committee. They are rather an exploration into what it would look like if TLS and DTLS were added without further changes.

The examples in this paper are based on using Chris Kohlhoff's networking TS implementation with checkout `c97570e7ceef436581be3c138868a19ad96e025b`. As an implementation detail, we use BoringSSL and a few APIs from `Security.framework` to access the platform's root store. An implementation of the changes have been published on the boost mailing list.

§ 3. Minimal Changes

The following changes are based on [N4771].

In Section 18.6, one new method should be added to Class template `basic_socket`:

```
class security_properties& security_properties();
```

Likewise in Section 18.9, one new method should be added to Class template `basic_socket_acceptor`:

```
class security_properties& security_properties();
```

A new section should be added, Section 22, entitled "Security", and containing initially just one class in Section 22.1 (entitled Class `security_properties`):

```
class security_properties {
public:
    using certificate_chain = std::vector<std::string_view>;

    security_properties& disable_security();
    security_properties& set_host(std::string_view);
    security_properties& use_private_key(std::string_view);
    security_properties& use_certificates(const certificate_chain&);

    template <typename Verifier>
        requires invocable<bool, Verifier, const certificate_chain>
    security_properties& use_certificate_verifier(Verifier);
};
```

The initial implementation uses PEM encoding from [RFC7468] for the `private_key` (in the `string_view`) and DER encoding for [X690] certificate chains (in the `vector`). A consistent and well-defined format for certificates and keys should be developed. The intent is to expand `security_properties` in future revisions of this paper so that it contains most if not all of the properties from a mature networking library, such as in `Network.framework`'s security options.

§ 4. TLS Client Example

Consider a simple TCP client that wants to fetch some data from the internet. It must first do a DNS lookup to get an IP address, then it should establish a connection, send a request, and receive a response:

```cpp
#include <array>
#include <experimental/net>
#include <iostream>
#include <string>

int main() {
    using namespace std::experimental::net;
    io_context io_context;

    // DNS lookup to get IP address
    ip::tcp::resolver resolver(io_context);
    const uint16_t port = 80;
    ip::basic_resolver<ip::tcp>::results_type results =
        resolver.resolve("www.apple.com", std::to_string(port));
    if (results.begin() == results.end()) {
        std::cerr << "error in DNS lookup\n";
        return 1;
    }
    ip::tcp::endpoint endpoint = results.begin()->endpoint();

    // Create TCP connection
    ip::tcp::socket socket(io_context);
    socket.connect(endpoint);

    // Send request
    std::string_view request = "GET / HTTP/1.1\r\nHost: www.apple.com\r\n\r
    std::error_code error;
    write(socket, buffer(request), error);
    if (error) {
        std::cerr << "error sending request\n";
        return 1;
    }

    // Receive response
    std::array<char, 1000> buffer;
    read(socket, std::experimental::net::buffer(buffer), transfer_at_least(
    if (error && error != error::eof) {
        std::cerr << "error receiving response: " << error.message() << '\r
        return 1;
    }

    std::cout << "received response:\n" << buffer.data() << '\n';
```

```
    return 0;
}
```

The changes in this paper would require one additional line of code in order to make a plaintext request:

```
ip::tcp::socket socket(io_context);
socket.security_properties().disable_security();
socket.connect(endpoint);
```

This particular server, `www.apple.com`, responds in plaintext only to redirect to the HTTPS version of the website. In order to make a secure request over TLS, two small changes are necessary: the port would need to be changed from `80` (the default port for HTTP) to `443` (the default port of HTTPS) and the `security_properties` would need to know what the intended host is in order to evaluate whether the TLS certificate used in the handshake is valid for this host:

```
const uint16_t port = 80443;
ip::basic_resolver<ip::tcp>::results_type results =
    resolver.resolve("www.apple.com", std::to_string(port));
// ...
ip::tcp::socket socket(io_context);
socket.security_properties().set_host("www.apple.com");
socket.connect(endpoint);
```

By default, the validity of the certificates will be evaluated by comparing the roots with the trusted roots on the system. This is the behavior most developers connecting to the internet would use. If a developer wants to allow deviations from this, they must use their own certificate verification function. This will allow use of self-signed certificates on the server, or connections to sites such as `wrong.host.badssl.com` which do not have trusted certificates that match the intended host:

```
socket.security_properties().use_certificate_verifier([] (const auto& cl
    return customCertificateVerifier(chain);
});
```

It should be understood that the use of custom certificate verification capability likely allows man-in-the-middle attacks, it should therefore be done with caution.

§ 5. TLS Server Example

Consider a simple TCP server that responds to one request with a fixed response. It must listen for a connection to a certain port, then when a client has connected it must read the request then send the response:

```cpp
#include <array>
#include <experimental/net>
#include <iostream>
#include <string>

int main() {
    using namespace std::experimental::net;
    using namespace std::experimental::net::ip;

    io_context context;

    ip::tcp::acceptor acceptor(context);
    ip::tcp::resolver resolver(context);
    const uint16_t port = 50000; // An unallocated port, likely to be unuse
    ip::basic_resolver<ip::tcp>::results_type results =
        resolver.resolve("0.0.0.0", std::to_string(port));
    ip::tcp::endpoint endpoint = results.begin()->endpoint();

    acceptor.open(endpoint.protocol());
    try {
        acceptor.bind(endpoint);
    } catch (...) {
        std::cerr << "binding failed\n";
    }
    acceptor.listen();
    std::cout << "try running 'curl http://127.0.0.1:" << endpoint.port()
        << "' in a terminal << std::endl;
    std::cout << waiting for connection" << std::endl;
    tcp::socket socket = acceptor.accept(context);

    std::cout << "waiting for request" << std::endl;
    std::error_code error;
    std::array<char, 1000> buffer;
    read(socket, std::experimental::net::buffer(buffer), transfer_at_least(

    std::cout << "writing response" << std::endl;
    std::string_view msg =
        "HTTP/1.1 200 OK\r\n"
        "Content-Length: 28\r\n"
        "\r\n"
        "<html><h1>hello!</h1></html>";
    write(socket, std::experimental::net::buffer(msg), error);
```

```
    return 0;
}
```

With these proposed changes, there will be one more required line of code to continue using plaintext:

```
  ip::tcp::acceptor acceptor(context);
  acceptor.security_properties().disable_security();
  ip::tcp::resolver resolver(context);
```

This is of course a bad idea: all connections to the server are now insecure. To set up a secure server, the only necessary steps are to add a certificate and a private key for the server to use in the TLS handshake:

```
    ip::tcp::acceptor acceptor(context);

    // This is a test certificate from
    // https://boringssl.googlesource.com/boringssl/+/2661/ssl/ssl_test.cc#
    // It is not signed by a trusted CA, which is why curl needs an
    // --insecure flag when communicating with it.
    acceptor.security_properties().use_certificates({ base64_decode(
        "MIICWDCCAcGgAwIBAgIJAPuwTC6rEJsMMA0GCSqGSIb3DQEBBQUAMEUxCzAJBgNV"
        "BAYTAkFVMRMwEQYDVQQIDApTb21lLVN0YXRlMSEwHwYDVQQKDBhJbnRlcm5ldCBX"
        "aWRnaXRzIFB0eSBMdGQwHhcNMTQwNDIzMjA1MDQwWhcNMTcwNDIyMjA1MDQwWjBF"
        "MQswCQYDVQQGEwJBVTETMBEGA1UECAwKU29tZS1TdGF0ZTEhMB8GA1UECgwYSW50"
        "ZXJuZXQgV2lkZ2l0cyBQdHkgTHRkMIGfMA0GCSqGSIb3DQEBAQUAA4GNADCBiQKB"
        "gQDYK8imMuRi/03z0K1Zi0WnvfFHvwlYeyK9Na6XJYaUoIDAtB92kWdGMdAQhLci"
        "HnAjkXLI6W15OoV3gA/ElRZ1xUpxTMhjP6PyY5wqT5r6y8FxbiiFKKAnHmUcrgfV"
        "W28tQ+0rkLGMryRtrukXOgXBv7gcrmU7G1jC2a7WqmeI8QIDAQABo1AwTjAdBgNV"
        "HQ4EFgQUi3XVrMsIvg4fZbf6Vr5sp3Xaha8wHwYDVR0jBBgwFoAUi3XVrMsIvg4f"
        "Zbf6Vr5sp3Xaha8wDAYDVR0TBAUwAwEB/zANBgkqhkiG9w0BAQUFAAOBgQA76Hht"
        "ldY9avcTGSwbwoiuIqv0jTL1fHFnzy3RHMLDh+Lpvolc5DSrSJHCP5WuK0eeJXhr"
        "T5oQpHL9z/cCDLAKCKRa4uV0fhEdOWBqyR9p8y5jJtye72t6CuFUV5iqcpF4BH4f"
        "j2VNHwsSrJwkD4QUGlUtH7vwnQmyCFxZMmWAJg==") });

    // This is a test key from
    // https://boringssl.googlesource.com/boringssl/+/2661/ssl/ssl_test.cc#
    .use_private_key(
        "-----BEGIN RSA PRIVATE KEY-----\n"
        "MIICXgIBAAKBgQDYK8imMuRi/03z0K1Zi0WnvfFHvwlYeyK9Na6XJYaUoIDAtB92\n
        "kWdGMdAQhLciHnAjkXLI6W15OoV3gA/ElRZ1xUpxTMhjP6PyY5wqT5r6y8FxbiiF\n
        "KKAnHmUcrgfVW28tQ+0rkLGMryRtrukXOgXBv7gcrmU7G1jC2a7WqmeI8QIDAQAB\n
        "AoGBAIBy09Fd4DOq/Ijp8HeKuCMKTHqTW1xGHshLQ6jwVV2vWZIn9aIgmDsvkjCe\n
        "i6ssZvnbjVcwzSoByhjN8ZCf/i15HECWDFFh6gt0P5z0MnChwzZmvatV/FXCT0j+\n
        "WmGNB/gkehKjGXLLcjTb6dRYVJSCZhVuOLLcbWIV10gggJQBAkEA8S8sGe4ezyyZ\n
        "m4e9r95g6s43kPqtj5rewTsUxt+2n4eVodD+ZUlCULWVNAFLkYRTBCASlSrm9Xhj\n
        "QpmWAHJUkQJBAOVzQdFUaewLtdOJoPCtpYoY1zd22eae8TQEmpGOR11L6kbxLQsk\n
        "aMly/DOnOaa82tqAGTdqDEZgSNmCeKKknmECQAvpnY8GUOVAubGR6c+W90iBuQLj\n
        "LtFp/9ihd2w/PoDwrHZaoUYVcT4VSfJQog/k7kjE4MYXYWL8eEKg3WTWQNECQQDk\n
        "104Wi91Umd1PzF0ijd2jXOERJU1wEKe6XLkYYNHWQAe5l4J4MWj9OdxFXAxIuuR/\n
        "tfDwbqkta4xcux67//khAkEAvvRXLHTaa6VFzTaiiO8SaFsHV3lQyXOtMrBpB5jd\n
        "moZWgjHvB2W9Ckn7sDqsPB+U2tyX0joDdQEyuiMECDY8oQ==\n"
        "-----END RSA PRIVATE KEY-----\n");

    ip::tcp::resolver resolver(context);

    // ...
```

```
    std::cout << "try running 'curl https://127.0.0.1:" << endpoint.port()
        << " --insecure' in a terminal" << std::endl;
```

A real server wouldn't use test keys. One would instead obtain certificates from a certificate authority such as Let's Encrypt.

§ References

§ Informative References

[N4771]

Jonathan Wakely. Working Draft, C++ Extensions for Networking. 8 October 2018. URL: https://wg21.link/n4771

[P1860R0]

C++ Networking Must Be Secure By Default. 2019-09-05. URL: https://wg21.link/P1860R0

[RFC7468]

S. Josefsson; S. Leonard. Textual Encodings of PKIX, PKCS, and CMS Structures. April 2015. Proposed Standard. URL: https://tools.ietf.org/html/rfc7468

[X690]

Recommendation X.690 — Information Technology — ASN.1 Encoding Rules — Specification of Basic Encoding Rules (BER), Canonical Encoding Rules (CER), and Distinguished Encoding Rules (DER). URL: https://www.itu.int/ITU-T/studygroups/com17/languages/X.690-0207.pdf